

Data Structures and Algorithms

Here's how I'd sequence your roadmap into phases so each one builds on the last:

1. **Foundations** — What data structures are, why they matter, and Big-O / asymptotic notation. I'd pull complexity analysis *forward* to here (the roadmap lists it later) because it's the vocabulary we'll use to reason about every structure and algorithm afterward. Without it, "why is this better?" has no precise answer.
2. **Linear structures** — Arrays, linked lists, stacks, queues, hash tables. The workhorses.
3. **Algorithms on linear data** — Sorting (bubble, insertion, selection, merge, quick, heap) and searching (linear, binary).
4. **Trees** — Binary trees, BSTs, traversals (BFS/DFS, in/pre/post-order), then balanced trees (AVL, B-trees).
5. **Graphs** — Representations, BFS/DFS, shortest paths (Dijkstra, Bellman-Ford), minimum spanning trees (Prim, Kruskal).
6. **Advanced & complex structures** — Tries, segment trees, Fenwick trees, union-find, and so on.
7. **Problem-solving techniques** — Recursion, divide & conquer, greedy, backtracking, DP, two-pointer, sliding window — then applying it all on LeetCode.

Phase 1: Foundations.

What a data structure is, and why it matters

A data structure is a way of organizing data in memory so that certain operations on it are cheap. Every structure is a set of tradeoffs — it makes some operations fast and others slow. An array gives you instant access by index but painful insertion in the middle; a linked list flips that. There is no "best" structure, only the best one *for the operations your problem performs most*.

Big-O is what lets you make that choice before writing a line of code. The same problem with the wrong structure can be thousands of times slower on large input, and Big-O predicts that gap.

Big-O: the core idea

Big-O describes how an algorithm's cost grows as the input size n grows, ignoring constant factors and hardware. We care about the *shape* of the growth, not the exact operation count, because for large n the shape dominates everything else.

There are three asymptotic notations, and knowing the difference separates people who really understand this from people who memorized it:

- **Big-O (O)** — the *upper* bound. "The algorithm grows no faster than this." Usually describes the worst case.
- **Big-Ω (Omega)** — the *lower* bound. "It grows no slower than this." Often the best case.
- **Big-Θ (Theta)** — the *tight* bound. When the upper and lower bounds match, Θ says "it grows exactly like this."

In everyday interview/engineering talk, people say "Big-O" but usually mean the tight worst-case bound.

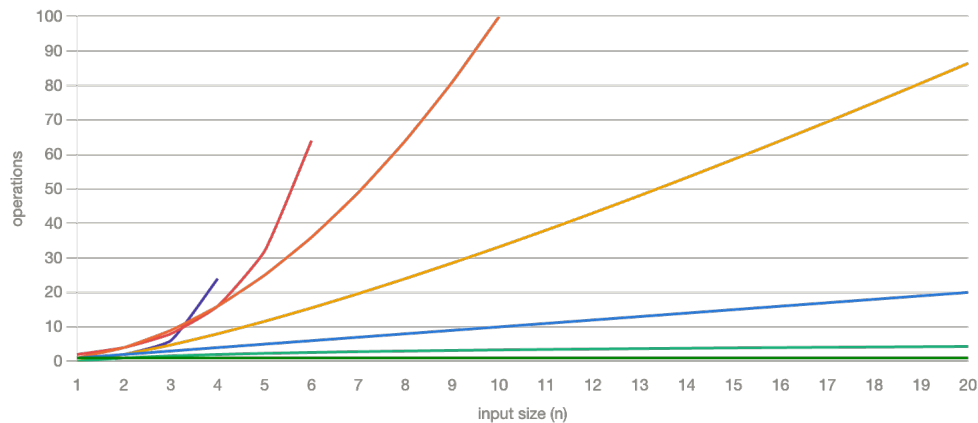
The common runtime classes

From fastest-growing (best) to slowest (worst), with the Java pattern that produces each:

- **O(1) constant** — `arr[i]`, a hash lookup. Input size doesn't matter.
- **O(log n) logarithmic** — binary search; you halve the problem each step.
- **O(n) linear** — one loop over n elements.
- **O(n log n)** — the efficient sorts (merge, heap, average quicksort).
- **O(n²) quadratic** — a loop nested inside a loop (bubble sort).
- **O(2ⁿ) exponential** — naive recursive Fibonacci, generating all subsets.

- **$O(n!)$ factorial** — generating all permutations, brute-force travelling salesman.

The reason we obsess over which class an algorithm falls into is that these don't grow "a bit" differently — they diverge violently:



Notice what the chart is really telling you: at $n = 20$, $O(\log n)$ is doing ~ 4 operations while $O(2^n)$ and $O(n!)$ have already blown off the top of the graph. The two flat lines at the bottom are the algorithms you *want*; the two that vanish upward are the ones that make your program hang. This is why the class matters far more than clever micro-optimizations — no amount of tuning saves an $O(2^n)$ algorithm at scale.

How to calculate Big-O

Three rules cover almost everything:

1. **Drop constants.** A loop that runs $2n$ times is still $O(n)$. Constants don't change the shape.
2. **Keep only the dominant term.** $O(n^2 + n + 5) \rightarrow O(n^2)$. For large n , the biggest term drowns the rest.
3. **Sequential code adds, nested code multiplies.** Two loops one after another $\rightarrow O(n) + O(n) = O(n)$. A loop inside a loop $\rightarrow O(n) \times O(n) = O(n^2)$.

A worked example in Java:

```
java
void example(int[] arr) {           // n = arr.length
    int first = arr[0];             // O(1) - one operation
    for (int x : arr) {              // O(n) - runs n times
        System.out.println(x);
    }
    for (int i = 0; i < arr.length; i++) { // outer: n times
        for (int j = 0; j < arr.length; j++) { // inner: n times each
            System.out.println(arr[i] + arr[j]);
        }
    }
}
```

Adding it up: $O(1) + O(n) + O(n^2)$. Drop the smaller terms $\rightarrow$ the whole method is **$O(n^2)$** .

Time vs space complexity

Everything above was *time* — how the operation count grows. **Space complexity** is the same analysis applied to extra memory the algorithm allocates, not counting the input itself. If you build a HashSet holding all n elements, that's $O(n)$ extra space. If you only use a couple of loop variables, it's $O(1)$ space regardless of n .

They often trade against each other: hashing spends $O(n)$ memory to buy $O(1)$ lookups. Recursion is a sneaky one — even if it allocates nothing, each pending call sits on the call stack, so recursion depth d costs $O(d)$ space.

That's the whole analytical toolkit. Now the "code it" step — here it's **read it and analyze it**. Give me the time complexity (and for #4, the space complexity too) of each, with a one-line reason:

```
java
// 1
int sum = 0;
for (int i = 0; i < n; i++)
    sum += i;

// 2
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        System.out.println(i * j);

// 3
int i = n;
while (i > 1)
    i = i / 2;

// 4
int[] copy = new int[n];
for (int k = 0; k < n; k++)
    copy[k] = k * k;
```

Phase 2, Topic 1: The Array

Theory first. An array is the most fundamental structure, and understanding *why* it behaves the way it does explains half of everything that comes later.

An array is a block of memory that is **contiguous** — all its elements sit right next to each other, and every element is the same size. That single physical fact determines all its performance characteristics:

- **Access by index is $O(1)$.** When you write `arr[i]`, the computer doesn't search. It computes `baseAddress + i × elementSize` and jumps straight there. Because every slot is identical in size and laid end-to-end, the address of any element is pure arithmetic. Index 0 and index 999,999 cost the same.
- **Search is $O(n)$.** If you don't know the index and want to find a value, you have no choice but to look at elements one by one. Worst case, it's the last one or absent — n comparisons.
- **Insert / delete in the middle is $O(n)$.** To insert at index 2 of a full-ish array, every element from index 2 onward must **shift** one slot over to make room (or shift back, to delete). On average that's $n/2$ shuffles $\rightarrow O(n)$.
- **Fixed size.** A raw array's length is decided when you create it. `new int[10]` is 10 forever. You cannot grow it.

That last limitation is a real problem — we rarely know how many elements we'll need up front. The solution is the **dynamic array**, which is exactly what Java's `ArrayList` is under the hood. The trick:

1. Keep a raw fixed array internally (the "backing array") with some spare capacity.
2. Track size (elements used) separately from capacity (slots allocated).
3. When you append and there's room, just drop it in size slot — $O(1)$.
4. When the backing array is **full**, allocate a new, bigger one (typically **double** the capacity), copy everything over, and continue.

That copy is $O(n)$ — but it happens rarely. Because we double, the expensive resizes get further and further apart, and the cost *averaged over many appends* works out to $O(1)$. That's called **amortized $O(1)$** , and it's a concept worth remembering.

Now you code it. Build a dynamic array from scratch in Java — don't use `ArrayList` or any `java.util` collection; a raw `int[]` as the backing store is the whole point. Implement a class, say `DynamicArray`, with:

- a backing `int[]`, plus size and capacity tracking,
- `void add(int value)` — append to the end, doubling capacity when full,
- `int get(int index)` — $O(1)$ access, throw on out-of-bounds,

- `void insert(int index, int value)` — insert at a position, shifting elements right,
- `int remove(int index)` — remove at a position, shifting elements left, return the removed value,
- `int size()`.

Phase 2, Topic 2: The Linked List

Theory first. The linked list is the array's mirror image. Where an array is one contiguous block, a linked list is a chain of separate objects — **nodes** — scattered anywhere in memory. Each node holds two things: a **value**, and a **reference (pointer) to the next node**. The list itself only holds a reference to the first node, called the **head**. The last node's next points to null, which is how you know you've reached the end.

head → [10 | •] → [20 | •] → [30 | null]

Because nodes are linked by references rather than sitting in a fixed layout, the performance profile *inverts* the array:

- **Access by index is $O(n)$.** There is no address arithmetic — nodes are scattered, so you cannot jump to the i -th one. You must start at head and follow next i times. This is called **traversal**, and it's the price of the whole structure.
- **Search is $O(n)$.** Same reason — walk the chain comparing values.
- **Insert / delete at the front is $O(1)$.** This is the linked list's superpower. To add a new head, you make the new node's next point at the old head, then move head to the new node. No shifting, ever — you just re-wire two pointers. The array *cannot* do this cheaply; prepending to an array is $O(n)$.
- **Insert / delete in the middle is $O(1)$ *once you're there*** — but reaching the spot is $O(n)$, so end-to-end it's $O(n)$.

The tradeoff in one sentence: **arrays are fast to read by index and slow to restructure; linked lists are fast to restructure at a known point and slow to read by index.** There's also a memory cost — every node carries an extra pointer, so a linked list uses more memory per element than an array.

One more critical idea before you code: **pointer manipulation order matters.** When you rewire nodes, if you overwrite a pointer before you've saved what it pointed to, you lose the rest of the list — it gets garbage-collected and is gone. You'll feel this the moment you write `insert`. The discipline is: *save the reference you're about to overwrite before you overwrite it.*

We'll build a **singly linked list** (each node points only forward).

Now you code it. No `java.util.LinkedList` — build it from scratch. You'll need an inner `Node` class (a value and a `Node next`) and a `SinglyLinkedList` class holding a head reference and a size.

Implement:

- `void addFirst(int value)` — new node becomes the head. $O(1)$.
- `void addLast(int value)` — append to the tail. $O(n)$ for now, since you'll traverse to find the end. (Handle the empty-list case: if head is null, this is the same as `addFirst`.)
- `int get(int index)` — traverse to the i -th node and return its value; throw on out-of-bounds.
- `void insert(int index, int value)` — insert at an arbitrary position. This is the pointer-order one. (`index == 0` is `addFirst`; `index == size` is `addLast`.)
- `int removeFirst()` — unlink and return the head's value; throw if empty.
- `int remove(int index)` — unlink the i -th node, return its value.
- `int size()`.
- a `toString()` that walks the chain, so you can print it like `10 -> 20 -> 30`.

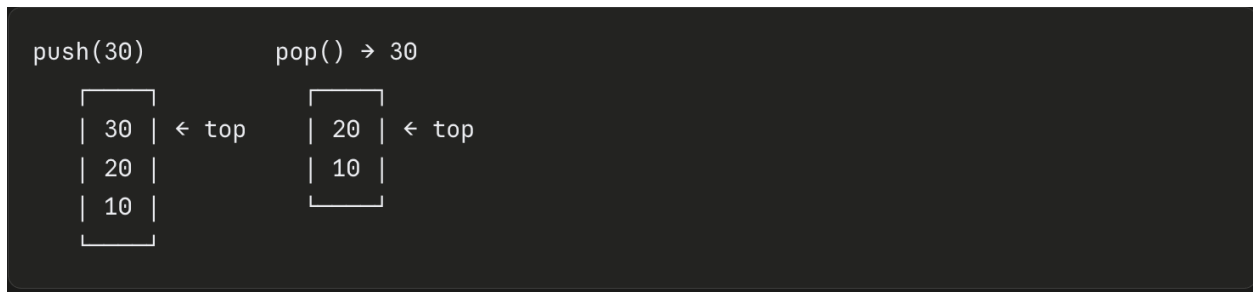
The tricky ones are `insert` and `remove` at a middle index: to unlink node i , you need a reference to node $i-1$ (the **predecessor**), because you rewire *its* next. Think about how to stop your traversal one node early. Take your time, write the whole class, and paste it — I'll review for correctness, pointer-order bugs, and the empty/single-node edge cases, which is where linked lists always break.

Phase 2, Topic 3: Stacks and Queues

Theory first. Here's the reframe that makes these easy: a stack and a queue are not new data structures. They're **access disciplines** layered on top of a structure you already built. All they do is *restrict where you're allowed to add and remove*. That restriction is the entire point — by giving up general access, you get a simple, predictable tool that models a huge class of real problems.

The Stack — LIFO (Last In, First Out).

A stack only lets you touch **one end**, called the **top**. You *push* (add to the top) and *pop* (remove from the top). The last thing you pushed is the first thing you pop — like a stack of plates: you take from the top, not the bottom.



Core operations, all **O(1)**: push, pop, peek (look at the top without removing), isEmpty. Where you see stacks in the wild: the **call stack** itself (every method call pushes a frame, every return pops it — this is literally why deep recursion throws `StackOverflowError`), undo/redo, browser back button, matching brackets in a parser, and depth-first traversal (which you'll meet in trees and graphs).

The Queue — FIFO (First In, First Out).

A queue lets you touch **both ends, but for different operations**. You *enqueue* (add at the **rear/tail**) and *dequeue* (remove from the **front/head**). The first thing you added is the first thing out — like a line at a checkout: you join at the back, you're served from the front.

enqueue → [10 | 20 | 30] → dequeue

(rear)

(front)

add here

remove here

Core operations, all **O(1)**: enqueue, dequeue, peek (front element), isEmpty. Where you see queues: task scheduling, print jobs, buffering, and **breadth-first traversal** (the counterpart to DFS, also coming in trees and graphs).

The implementation insight — this is what I want you to internalize. Both can be built on either an array or a linked list, and the choice reveals *why* we built the linked list the way we did:

- A **stack** is trivial on your `DynamicArray`: push = add at the end, pop = remove from the end. Both are the array's cheap $O(1)$ operations. No shifting, because you only ever touch the last slot.
- A **queue** is the interesting one. On an array, dequeue removes from the *front* — which is the array's expensive $O(n)$ shift-everything operation. But on your **linked list**, if you remove from the head and add at the tail, both are $O(1)$ — *provided you kept a tail pointer*. Which you did. So a linked list with head-and-tail pointers is the natural backing store for a queue.

That's the connection I want you to see: the tail pointer you added on your own initiative last topic is *exactly* what makes an efficient queue possible.

Now you code it — two small classes. Build both from scratch, no `java.util.Stack`, `ArrayDeque`, or `Queue`:

1. **Stack** — back it with an `int[]` (your own, or the same growing-array idea from `DynamicArray`; a plain resizing array is fine). Implement `void push(int value)`, `int pop()`, `int peek()`, `boolean isEmpty()`, `int size()`. `pop` and `peek` must throw on an empty stack.
2. **Queue** — back it with a **linked list using head and tail pointers** (reuse the `Node` idea). Implement `void enqueue(int value)`, `int dequeue()`, `int peek()`, `boolean isEmpty()`, `int size()`. `dequeue` and `peek` throw on empty. The whole goal is that **every** operation is $O(1)$ — if you ever find yourself writing a traversal loop, something's off.

Watch the edge cases that always break these: popping/dequeuing the *last* element (your stack's top index or your queue's head/tail must end up in a valid empty state), and calling `pop/dequeue` on an empty container. Write both, test them — including draining to empty and then pushing again — and paste them. I'll review both, then we've cleared all of Phase 2's linear structures and move into sorting.